Complete implementation and teaching plan

Project: MAXX Bytes campus delivery demonstration · **Partners:** David and Shek · **Prepared:** October 26, 2026 · **Demo:** October 27, 2026.

1. What this card demonstrates

Customer need: staff can see campus GIS layers, inspect valid demonstration routes, and track a simulated robot with useful arrival estimates on desktop and phones.

Acceptance item	Implemented behavior	Owner for review	Evidence still needed
Position updates every 10 seconds	Browser polls the shared Python simulation; selectable 1-second enhancement	Shek	Observe timestamps during a local trip
Users zoom and pan	Leaflet drag, zoom controls, wheel and touch gestures	David	Desktop and device gesture tests
Users see ETA	Time to delivery outbound; time to base returning; paused and unavailable states	David	Screenshot plus observed countdown
iOS and Android	Responsive viewport, wrapping controls, touch-capable map	David	Physical Safari/iPhone and Chrome/Android sign-off
Incoming GIS is simulated	Timestamped, labeled coordinates from route-distance progression	Shek	Two viewers see the same trip
Team GIS overlays	Snapshot walkways/restrictions plus 145 supplemental OSM buildings and simulated delivery points	David	Review provenance and simulated-delivery labels
Valid routes avoid blocked places	Dijkstra excludes closed edges; forecast return blockage can reroute; sealed destination has no route	Both	Closure/no-route demonstration
Weighted heat	Intensity, confidence, freshness, time window, distance falloff	Both	Heat scenario and explanation

The route-as-simulated-robot concept is credited to Shek, who gets the first opportunity to own and extend the telemetry implementation. This kit proposes ownership; it does not change your team's assignments or publish anything.

2. Architecture and why each part exists

The browser fetches map/configuration once. It then fetches `/api/state` every 10 seconds, or roughly every second when near-live is selected. The Python server advances one shared simulated trip using elapsed monotonic time. Dijkstra selects the route from the team export's node connections. Leaflet draws the overlays and robot.

Why a Python server instead of a static file server? The required shared robot, route calculation and controls need an API. A plain `python -m http.server` only serves files and would return 404 for `/api/state`. This kit

uses Python's standard HTTP classes and no framework to keep installation simple.

Why server-owned time? If each phone ran its own independent simulation, start times and positions could disagree. Here, a viewer joining halfway through receives the current shared trip. Polling reads state; it does not add a fixed distance per poll. This prevents faster polling or a second viewer from making the robot move faster.

Why monotonic time? Travel progression uses a clock that measures elapsed time independently of wall-clock adjustments. Human-readable timestamps use UTC wall time. The robot moves while the process is running even if no browser is watching; restarting the server resets the demonstration.

Files and responsibilities

File/folder	What to study or change
`server.py`	HTTP endpoints, static web root, request validation and shared-state locking
`engine.py`	Graph import, hard exclusions, weighted Dijkstra, exact or rerouted return and simulator
`web/index.html`	Page structure, labels and controls; David's main presentation area
`web/style.css`	Desktop/phone layout, control sizes, map height and status styling
`web/app.js`	Leaflet layers, heat canvas, position polling, ETA and staleness display
`web/vendor/`	Pinned Leaflet 1.9.4 and license, served locally
`data/gmu_map_16OCT26.json`	Original team OSM export with node IDs/geometry/tags
`data/campus-layers.geojson`	Team's display-ready walkway/restriction export
`data/buildings.geojson`	145 supplemental real OSM building polygons with source metadata
`crowd_model.py`	Supplied validated weight/freshness/kernel and edge-exposure model
`web/heat-field.js`, `web/heat-layer.js`	Matching field calculation and sampled canvas heat
`lessons/`	Four short programs to type and explain in PyCharm
`data/SOURCE_MANIFEST.json`	Exact source URLs, commit, hashes, source dates and limitations
`tests/test_demo.py`	Reproducible automated acceptance checks
`docs/`	Teaching, acceptance evidence and code reference

The bundled earlier map data is read-only input. This new engine does not import or modify the team's backend algorithm. It implements a small Dijkstra adapter using their original node IDs. It does not require the earlier dynamic/weighted ZIPs; no previous phase is deleted. Those older packages were not assumed to be installed.

3. Windows and PyCharm implementation, step by step

Step 1 - Unzip and inspect

Download the ZIP and use **Extract All**. Confirm that `server.py` sits next to `engine.py`, `data`, and `web`. Running inside a compressed-folder window can break relative paths.

In PyCharm choose **File -> Open**, select the extracted `MAXX_David_Shek_Demo` directory, and open it as a project. Select an existing Python 3.10+ interpreter in project settings. No new packages are required. If your existing project already has a working `.venv`, its Python can run these files.

Open **Terminal** at the bottom. Enter, pressing **Enter** after each line:

```
python --version
dir
```

If Python is not found, use `py -3 --version`. Confirm the version and the presence of `server.py`; do not paste a different project's path into the commands. If neither Python command exists, install Python from python.org/downloads with the launcher and PATH options, reopen PyCharm, and retry.

Step 2 - Start this server

Stop the old static port-8000 server with Ctrl+C in its own terminal. In the kit terminal run:

```
python server.py
```

Press **Enter**. Leave it running. Open the local demo. You can also open the short address. Both serve this kit's `web/index.html` through an alias; the original repository page remains intact.

The included `.cmd` launchers change into their own folder before starting Python. If `py` exists, they use its Python 3 interpreter; otherwise they use `python`.

Step 3 - Establish a baseline

Select delivery 2, clear conditions and 1x time. Apply the configuration, then choose 10-second updates and start. Record the first position time, then observe subsequent updates. The polling delay is approximately 10 seconds plus the local request time; it is not a hard real-time guarantee.

Switch to 1-second updates to show the improvement. This is near-live simulated telemetry; it is not GPS, a WebSocket, or a physical robot connection. Animation updates on received samples rather than inventing movement between samples.

For a short talk, choose 10x playback and explicitly tell the audience it is accelerated. Model speed remains 1.2 meters per model second. ETA is divided by 10 to match demo wall time; the API also exposes unscaled travel seconds.

Step 4 - Test and interpret

Open a second terminal without stopping the server. Run:

```
python -m unittest discover -s tests -v
```

Press **Enter**. Twenty-five Python tests should pass. These checks verify six-stop routing, blocked-edge exclusion on both legs, no-route behavior, movement, timing, ETA, weight decay, and HTTP response contracts. They do not prove real-world map accuracy or actual iOS compatibility. Record physical device results separately.

4. David's implementation lessons

The map and layers

`L.map(...)` creates a single persistent map object. Creating a new map at each update would reset zoom, pan and layer choices. Instead, `setLatLng()` changes the robot's location and `setLatLngs()` changes route

lines only when a scenario revision changes.

`L.geoJSON(...)` understands standard GeoJSON longitude-first coordinates. Leaflet marker and polyline helpers expect latitude-first pairs. `latlngs()` is the explicit conversion boundary. Mixing those orders would send the map far outside Virginia.

The layer control keeps independent overlay groups. Walkways are green; real supplemental OSM buildings are blue; sample deliveries are maroon; restrictions and demo closures are red; the outbound route is purple; the return is a thin teal dashed line. Under ordinary conditions it retraces the outbound geometry; under forecast return closure it uses its planned alternative. Feature popups and walkway tooltips use plain text, including OSM identifiers for unnamed paths. No walkway names are invented.

The offline background deliberately does not request internet tiles. All supplied vector layers remain visible. The optional online OSM background provides place names and context, but it is not an independent positional reference because the team overlay also comes from OSM.

ETA

Each edge has a length and estimated travel duration. The simulator knows how many model seconds have elapsed. Outbound ETA is `outbound_duration - elapsed`. On the return it becomes `round_trip_duration - elapsed`. At arrival at base, ETA is zero. If no path exists, ETA is null and the UI says Unavailable.

Pause freezes model time. An ETA marked paused is remaining duration, not a promised clock-time arrival. There is no handoff/delivery dwell in this demo. To add one later, introduce a separate dwell phase and include it in total trip duration.

Mobile behavior

The viewport meta tag uses the device width. CSS wraps controls and changes four metric columns into two. Buttons have a minimum 44-pixel height. The map reserves a visible height; without it, Leaflet may appear blank. Native Leaflet gestures support touch inspection. A phone-size desktop emulation is useful layout evidence, but it cannot substitute for an iPhone and Android test.

5. Shek's implementation lessons

GIS graph

The original JSON contains OSM ways with `nodes` and `geometry`. Shared node IDs connect segments; merely crossing visually does not create a connection. A bridge and path may cross on a map while remaining separate. This is why the engine does not connect arbitrary nearby coordinates or draw straight lines between requested destinations.

For this local pedestrian demonstration, allowed types are footway, pedestrian, path and cycleway. Steps, roadways, polygon areas and ways tagged `access=no/private` or `foot=no/private` are excluded. This is a limited demonstration policy, not a complete robot-access model. Width, slope, gates, barriers, surface, construction, crossing permission, pedestrian one-way rules and campus operations need further review. The display layer may contain paths that this policy excludes; display visibility never overrides route eligibility.

The base is snapped to the graph near the earlier starting coordinate. Six stops are selected from its connected component. This deliberately differs from the uploaded older road-inclusive planner: all six here are connected pedestrian demonstration stops, so they may snap away from their requested targets. The old saved endpoint is a target, not a guarantee of an exact approved entrance. Their requested coordinates, snapped coordinates and offsets are exposed by `/api/config`. These are not verified dining-hall entrances or a ranking of the largest dorms. Do not relabel them as such without evidence.

Dijkstra and hard closures

Dijkstra uses a priority queue. It repeatedly expands the node with the lowest known estimated cost until reaching the goal. A closure removes the segment from relaxation entirely; it does not merely add a penalty that could still be chosen. The sealed scenario closes every incident edge at the selected goal. No path is returned, Start is disabled, and the robot stays at base.

The ordinary closure scenario chooses one edge of the baseline route whose removal still permits an alternative where possible. All six included destinations were checked. A scenario changes the shared configuration and resets to base; this is a deterministic demonstration of replanning before departure. It does **not** safely handle an obstacle appearing underneath a moving robot. Production must stop or safely complete its current segment, refresh conditions and replan from a validated current position.

Simulated incoming positions

`Simulation.snapshot()` uses elapsed time to locate the current route segment. The coordinate is linearly interpolated within that short segment. It never jumps directly from the starting point to the destination or crosses a blocked shortcut. The outbound route is traversed, then the validated return path leads back to base. Ordinary scenarios retrace exactly; the forecast-return closure scenario uses an explicitly rerouted return.

`/api/state` includes coordinates, timestamp, robot ID, mode, phase, remaining duration, trip distance and routes. The server uses one lock around mutations and reads so two viewers cannot corrupt the shared trip. This is a single-robot demonstration with no persistent database or per-user isolation.

Actual telemetry handoff, after the demo

Keep the interface stable and replace the simulation behind `/api/state` with an adapter for the final approved GPS/GIS source. Validate longitude/latitude range, robot identity, timestamp freshness, sequence order and accuracy. Reject malformed or older samples. Separate planned route from observed location. Do not relabel predicted/interpolated locations as observed GPS.

A useful real-position payload is:

```
{"robot_id":"ROBOT-01","mode":"observed","coordinates":[-77.308,38.831],"timestamp":"2026-10-
27T14:00:00Z","accuracy_m":2.5,"sequence":42}
```

This is a proposed contract, not an active hardware endpoint. The bundled server accepts demo controls only; it deliberately has no telemetry ingestion endpoint. A final authenticated feed, stale-position stop policy and route-progress matching remain integration work.

6. Weighted crowd points and heat

The point's effective weight is `intensity x confidence x freshness` within its active time window. The supplied schema names intensity `weight`. Freshness is `2 ** (-age_seconds / (60 * half_life_minutes))`. Spatial influence is `max(0, 1 - distance/radius_m) ** 2`. Overlapping sources combine as `1 - product(1 - contribution)`, bounded at one. These are the same formulas in the supplied crowd model and JavaScript field renderer.

The classroom example uses three sources with different intensities and confidence values, a 100-meter radius, a one-hour active window and a 60-minute freshness half-life. The first source has intensity 0.95 and confidence 0.8, giving effective center weight 0.76 at update time before overlapping contributions. These are arbitrary teaching assumptions, not a calibrated prediction of students per square meter. No private student schedules are included.

For each routing segment, the engine samples at intervals no longer than 10 meters, then calculates a length-weighted mean crowd exposure:

```
travel_seconds = segment_meters / 1.2 * (1 + 3 * crowd_score)
```

Zero influence leaves the base travel time. Full influence multiplies it by four. A node's score is not also charged as a separate node penalty: doing both could double-count the same crowd delay. The heat canvas uses matching weight/freshness/radius inputs but the canvas evaluates the same field at a 12-pixel grid. This is a sampled rendering on a fixed 0-1 scale, not exact surveyed boundaries.

Crowd costs and heat use the same planning-time snapshot for consistent route and ETA playback. Freshness is evaluated again when Apply creates a new snapshot; neither routes nor heat silently change while playing a trip. Ordinary closures apply on both legs. The forecast return-closure scenario separately blocks a segment on the return leg, computes an alternative and labels it **rerouted return**. If no valid return exists, the robot must stay at base. A future live system must reevaluate real conditions at expected return departure; the supplied return-closure scenario is an explicit synthetic snapshot, not a campus forecast feed.

Later data sources and refresh plan

Data	Possible owner/source to obtain	Suggested starting refresh	Treatment
Closures/construction	Campus facilities-approved feed or staff updates	On change, with expiry	Hard routing exclusions
Class changes	Approved aggregated registrar schedule	Import every 12-24 hours; evaluate time windows each cycle	Predicted weights
Campus/outside-group events	Approved events export	Import every 12-24 hours plus cancellations	Predicted weights
Observed crowd counts	Approved counters or staff observations	According to timestamp/TTL	Observed weights with confidence
Robot telemetry	Final team-approved source	Required 10 seconds; faster if supported	Observed position with age/accuracy

These are future integration targets, not verified accessible feeds or scheduled jobs in this kit. The running demo only reevaluates its local simulator when requests arrive. A production importer needs source authorization, schema validation, atomic snapshots, expiry, monitoring and a recorded owner. Unknown/expired restrictions must have an explicit operations policy; silently treating stale restrictions as open is unsafe.

7. APIs to inspect together

Open configuration, map layers and current robot state to see the data the frontend receives.

Endpoint	Method	Purpose
`/api/config`	GET	Six stops, base, source commit and graph counts
`/api/layers`	GET	Team vectors plus 145 supplemental OSM building polygons
`/api/state`	GET	Current shared simulated trip and timestamp
`/api/configure`	POST	JSON `stop`, `scenario`, `rate`; reset trip to base
`/api/action`	POST	JSON `action`: start, pause or reset

PowerShell example, while the server runs in the other terminal:

```
Invoke-RestMethod 'http://localhost:8000/api/state'
Invoke-RestMethod -Method Post -Uri 'http://localhost:8000/api/configure' -ContentType
'application/json' -Body '{"stop":"2","scenario":"closure","rate":10}'
Invoke-RestMethod -Method Post -Uri 'http://localhost:8000/api/action' -ContentType
'application/json' -Body '{"action":"start"}'
```

Press Enter after each command. Configure returns a complete reset state. Each browser's next poll receives it. A normal refresh does not reset the server's robot. Server restart does reset it.

8. Phone setup and compatibility evidence

Follow START_HERE for `--host 0.0.0.0`, the laptop's actual Wi-Fi IPv4 address and the same-network requirement. On the laptop `localhost` is correct; on phones it is not. This local demo uses HTTP and does not request device GPS, camera or other secure-context permissions.

Windows firewall can allow Python for a private-network demo. Do not open unrelated ports or disable the firewall. If it still fails, confirm the server bind address, the IP, port and whether the Wi-Fi isolates clients. A trusted personal hotspot is often simpler than campus guest Wi-Fi.

On each physical device record model, OS, browser, screen orientation and test outcome. Test map drag/pinch, layer selection, ETA visibility, moving position, a paused trip, no-route state, and rotate portrait/landscape. Use the checklist. Do not mark the card's physical compatibility criterion complete until these tests are observed.

9. Review and optional Git submission

For tomorrow's demo, running the extracted ZIP is sufficient. There is no need to SSH into a production server or publish a site. This kit has not changed the repository.

For an optional installer, run `powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\Install.ps1 -Repository 'C:\Users\noyes\PycharmProjects\MAXX-Map-Submission'` from the kit folder after reviewing that script. Correct that repository path if it differs on your laptop. It requires the expected origin and `human-only` branch, refuses an existing destination, and copies into one new folder without Git writes.

For a later reviewed submission, use the human-only branch. Open your existing `MAXX-Map-Submission` project and run:

```
git status --short
git branch --show-current
```

Press Enter. If uncommitted work exists, resolve or save it deliberately before switching/pulling. Once your workspace is clean, switch to the existing branch and fast-forward:

```
git switch human-only
git pull --ff-only origin human-only
```

Copy the extracted kit folder into the repository root as `MAXX_David_Shek_Demo`. If that name already exists, compare and back it up first; do not overwrite it blindly. Keep original `html/web/public/dave-campus-map/gis-layers.html` unchanged. Run the kit server from its new folder and repeat acceptance checks. This is an additional local demonstration folder, not integration into the deployed web server.

From the repository root, review only the addition:

```
git status --short
git add -- MAXX_David_Shek_Demo
git diff --cached --stat
```

```
git diff --cached -- MAXX_David_Shek_Demo/server.py MAXX_David_Shek_Demo/engine.py
MAXX_David_Shek_Demo/web
```

Have David and Shek review the staged changes and test results before committing or pushing. When that review is approved, the person doing the submission can run:

```
git commit -m "Add David and Shek local campus robot demonstration"
git push origin HEAD:human-only
```

Do not run the push command against another branch. If a push is rejected, fetch and review the new team changes; never force-push. This ZIP does not execute Git commands automatically or carry approval from another conversation.

10. Troubleshooting and limits

Symptom	What to do
Old screenshot/page still appears	Stop old static server; start this kit's server; verify `/api/config` loads; Ctrl+F5
Address already in use / WinError 10048	Stop old server in its terminal, or use `--port 8004` and update URL
API returns 404	Wrong server; a static server cannot run these endpoints
Page blank or controls stay disabled	Confirm local vendor files extracted; inspect console/network; check `/api/layers`
No street tiles	Choose optional online tiles and check internet; offline GIS remains functional
Tracking-prevention messages	The screenshot showed warnings while the map still rendered. This kit serves Leaflet locally, removing the unpkg runtime dependency; no browser tracking settings need changing
Some buildings are missing	The bundled 145 supplemental outlines cover a central-campus bounding box and simple closed ways only; multipolygon relations are omitted
Robot position appears frozen	Check Pause, no-route, trip complete, selected 10-second cadence and connection warning
Phone cannot connect	Bind to `0.0.0.0`, use laptop IPv4, same Wi-Fi, private-network firewall permission
Two people change the trip	Controls are deliberately shared; nominate one driver for the presentation
Need exact current return conditions	Not part of the fixed-snapshot demo; refresh and validate at return departure in the production design

Current scope does not include actual hardware navigation, live crowds, imported campus schedules, independent one-meter accuracy validation, independently surveyed complete building coverage, authenticated multi-user controls, persistent storage, production hosting or iOS/Android native apps. These are concrete remaining integrations, not hidden dependencies for tomorrow's local demonstration.

11. Small practice tasks

David: change only the heading text, reload, and explain why it does not change the robot coordinates. Then toggle buildings off, pan/zoom, and wait for a position update to verify those choices persist.

Shek: inspect `/api/state` before starting and again after 10 seconds of movement. Identify coordinate order, mode and timestamp. Pause for 20 seconds and verify the distance is unchanged. Change the scenario and explain why the new revision resets to base.

Together: compare clear and closure scenarios for stop 2. Explain why the red segment cannot appear in either trip leg. Select sealed access and show that a route finder can demonstrate failure honestly without drawing a straight-line substitute.

Learning order and source reference

Read **TEACHING_GUIDE.md** for short programs to type into the PyCharm editor, the exact terminal command for each, expected output and an explanation to rehearse. Read **DATA_DICTIONARY.md** for every crowd/telemetry field. **CODE_REFERENCE.md** contains the complete runnable source, not placeholders. The earlier uploaded weighted server is not required to run this standalone kit; source integration and production scheduling are explicitly future work.

Learn it by typing: editor versus terminal

The working app is supplied in full. First run it, then use these short lessons to understand it. Practice in a **copy** of the extracted kit so an unfinished edit cannot interrupt the demonstration.

Where each kind of code goes

What you see	Where to put it	How to run it
Python: `print(...)`, `def ...`, `import ...`	A `.py` file in the PyCharm editor	Right-click file -> Run, or `python filename.py` in Terminal
PowerShell: `python ...`, `git ...`, `Set-Location ...`	PyCharm's Terminal tab, with PowerShell selected	Paste the command and press Enter
HTML: `<div>`, `<button>`	`web/index.html` in editor	Start Python server, open browser and reload
CSS: `body { ... }`	`web/style.css` in editor	Reload browser; Ctrl+F5 if cached
JavaScript: `fetch(...)`, `L.map(...)`	`web/app.js` in editor	Browser runs it after loading the page
JSON / GeoJSON	The corresponding `.json`/`.geojson` file in editor	Python or JavaScript reads it; do not run it as a program

Do not paste the Markdown backticks or terminal prompts into the editor. Python indentation belongs to the program: preserve it. Every code block in CODE_REFERENCE.md is labeled with its destination file and can be copied without line-number cleanup.

Lesson 1 - Make a Python file

In PyCharm open the kit folder, right-click `lessons` -> New -> Python File. To type from scratch, name your file `my_01_hello.py` so you keep the supplied answer file intact. Type the contents of `lessons/01_hello.py`, which are also in CODE_REFERENCE.md. Save with Ctrl+S.

In Terminal, with the kit directory as the current folder, enter:

```
python lessons/my_01_hello.py
```

Press Enter. Expected output: the project name followed by a dictionary with robot ID and coordinates. Explain it aloud: “This is a record representing a position; it is not moving a real robot.”

If using the supplied file instead, run `python lessons/01_hello.py`. You do not need to install anything.

Lesson 2 - Calculate ETA with units

Type `lessons/02_eta.py` into `lessons/my_02_eta.py`. Run `python lessons/my_02_eta.py` and press Enter. Expected outputs are 100 model seconds and 10 presentation seconds. Change the distance from 120 to 240; both should double. Explain why a 10x clock does not mean the model robot is approved to drive ten times faster.

Lesson 3 - Explain weights and freshness

Type `lessons/03_weighted_point.py` into your own copy. Run it from Terminal. Expected weights: 0.4 initially and 0.2 after an hour. Change only confidence from 0.5 to 1.0; the results become 0.8 and 0.4.

Restore your practice copy if needed.

`validate_point` checks required fields and numeric ranges. `timestamp` rejects missing time zones. `effective_weight` applies the time window and freshness. This is the supplied crowd model, not a separate teaching approximation. Dates in this exercise are fixed so its answer remains reproducible.

Explain it: “The weight is an intensity estimate adjusted by trust and age. It is not a measured student count. Distance influence is calculated separately.”

Lesson 4 - Check a real graph route

Study/type `lessons/04_route_check.py`. Run `python lessons/04_route_check.py`. It reads local data, constructs a closure scenario, and checks each adjacent route edge. Expected: `Outbound avoids closure: True`, then the return kind and a distance. The test does not claim the route is physically approved.

Try `robot.configure('2', 'sealed', 1)`. In the full app that scenario gives no trip and disables Start. An `all(...)` check over an empty list is true in Python; therefore a **complete** availability check also requires a nonempty path. The app and automated tests check both. This is a useful reason to test failure states, not just happy paths.

Lesson 5 - Read and explain the full engine

Open `engine.py`. Follow these units in order:

1. `meters` and `edge_key`: coordinate units and stable segment identity.

2. `Campus.__init__`: read team ways, preserve shared node identity, reject unsuitable categories, select connected demonstration stops.

3. `Campus.duration`: sample crowd exposure along each segment and convert meters to seconds once.

4. `Campus.route`: Dijkstra relaxes unblocked edges, then reconstructs the path.

5. `Simulation.configure`: create the selected scenario and plan both legs before movement.

6. `tick`, `action`, `snapshot`: elapsed-time progression, pause/reset and position/ETA response.

You may retype one function at a time into a practice copy. Restore it from CODE_REFERENCE.md if indentation or spelling becomes confusing. Run the test suite after every change to routing or time progression.

Lesson 6 - Understand the server

Open `server.py`. `make_handler` shares the simulation and a lock among requests. GET reads state; POST changes state. The static root is only `web/`, so Python source and operator data are not exposed as web downloads. The familiar map URL is an alias to `web/index.html`; no original map file is replaced.

Terminal commands to run your practice copy are the same as the complete app:

```
python server.py
```

Press Enter. Leave it running. Open the map in the browser. Use a second terminal for tests or API inspection. Use Ctrl+C in the server terminal to stop it before editing/restarting Python code. Browser reload applies frontend edits; Python edits need a server restart.

Lesson 7 - David's browser code

Read `web/index.html`, then `web/style.css`, then `web/app.js`. The HTML names elements, CSS arranges them, and JavaScript updates their content. `$()` selects an element by ID. `request` handles HTTP and a timeout. `show` updates existing layers instead of rebuilding the map.

Safe exercise: change the introductory paragraph in HTML, save, and reload. Next change a color in CSS. Restore your practice copy afterward. Do not paste those changes into Terminal.

`heat-field.js` calculates the same crowd field as Python. `heat-layer.js` samples it on a canvas. Source and routing-node marker overlays are explanations; turning them off does not change route costs.

Lesson 8 - Shek's telemetry handoff

Read `/api/state` while the simulator moves. Find coordinates, timestamp, mode, phase and ETA. The frontend's polling cadence controls how often it reads positions. The backend clock controls movement. A future real-source adapter should replace the simulator without changing those meanings. Keep `mode: observed` separate from simulated or predicted positions.

Before presenting

Use DEMO_CHECKLIST.md. Both partners should be able to explain one successful route, one blocked route, one unavailable trip and how confidence/freshness affect heat. Run tests in the complete kit, then rehearse using its untouched code.

Data dictionary

Coordinates and identifiers

Field	Meaning / units	
`coordinates`	`[longitude, latitude]` in decimal degrees, WGS84; Leaflet helpers use reversed order	
`node` / `node_id`	OSM node identity used to connect paths; crossing lines are not automatically connected	
`way_id` / `osm_id`	Source OSM feature identifier	
edge key	Sorted endpoint node IDs joined by `` ` ``; this demo closes a graph segment	
`requested_coordinates`	Original demonstration target, not a verified entrance	
`snap_meters`	Distance from that target to the selected connected graph node	
layer	`walkways`, `buildings`, `delivery` or `restricted`	

Weighted crowd point schema (supplied model)

Field	Type / range	Meaning
`id`, `name`, `source`	Nonempty strings	Record identity, display name, provenance
`kind`	`class`, `event`, `manual`	Source category
`data_mode`	`simulated`, `predicted`, `observed`	Truthful evidence classification
`lat`, `lon`	Finite geographic degrees	Point center
`radius_m`	1-500 meters	Influence reaches zero at this radius
`weight`	0-1	Intensity estimate; this is the field name used for intensity
`confidence`	0-1	Trust factor applied to intensity
`half_life_minutes`	1-10080 minutes	Freshness halves after this age
`starts_at`, `ends_at`	ISO 8601 with zone	Active interval, inclusive start and exclusive end
`updated_at`	ISO 8601 with zone	Source update time, used for freshness
`effective_weight`	Derived 0-1	Intensity x confidence x freshness at planning time

Kernel: `max(0, 1 - distance/radius)^2`. Combine contributions as `1 - product(1 - contribution)` so overlap remains bounded. This is a relative crowd index, not people per square meter or a probability. Sample each walkway at no more than 10-meter spacing; use length-weighted exposure for its time cost. There is no additional routing-node penalty.

/api/state

Field	Meaning
`robot_id`, `mode`	`SIM-01`, `simulated`; no real hardware dispatch
`timestamp`	UTC time state was generated, not a GPS observation time
`planned_at`, `revision`	Scenario planning timestamp and monotonically increasing configuration version
`running`	Whether the model clock is advancing
`phase`	`outbound`, `returning`, `complete`, `unavailable`
`rate`	1 or 10; model seconds per wall second
`eta_seconds`	Remaining demo wall seconds to delivery or base; null when unavailable
`eta_travel_seconds`	Remaining unscaled model travel seconds
`eta_target`	`delivery` outbound, `base` on return/completion
`traveled_meters`, `total_meters`	Actual modeled trip distance, not cost units
`outbound_seconds`, `total_seconds`	Full unscaled model travel durations
`outbound`, `return_route`	Ordered arrays of longitude/latitude positions
`return_kind`	`exact retrace`, `rerouted return` or `unavailable return`
`blocked`	Active outbound closure segments, also considered for return
`forecast_return_blocked`	Additional segments closed only in the return planning snapshot
`crowds`, `weighted_points`	Raw scenario crowd records and planning-time prepared records
`weighted_nodes`	Node identity, coordinates and score for optional display; never a second routing cost

Units and limits

Base model speed is 1.2 m/s. Maximum crowd cost multiplier is four. These are demonstration assumptions, not legal or campus operating limits. The kit does not enforce a 20-minute service guarantee or blanket 15 mph. There is no delivery dwell. Return closures are explicit scenario snapshots; this is not continuous obstacle detection or a forecast feed.

David + Shek: October 27 presentation and sign-off

Before the demo

- [ ] ZIP fully extracted, Python interpreter works.

- [ ] Original port-8000 server stopped; this kit's server running.

- [ ] `/api/config` returns six stops and the pinned source commit.

- [ ] Automated tests pass locally.

- [ ] Choose offline GIS background for a dependable baseline; enable online tiles only if available.

- [ ] Nominate one control operator: changes affect all viewers.

- [ ] Explain that movement, deliveries, closures and crowds are simulations; buildings are supplemental OSM outlines.

Five-minute presentation

Time	Driver	What to show / say
0:00-0:40	David	Show the campus overlay and source labels. Toggle walkways, blue OSM buildings, maroon sample deliveries and restrictions.
0:40-1:15	David	Drag/pan and zoom; show that a position update does not reset the view.
1:15-2:00	Shek	Clear delivery 2, 10x clock, 1-second updates; Start. Explain route-as-simulated-robot ownership.
2:00-2:40	David	Show ETA, distance and Pause/resume. Explain demo-clock ETA; show a phone watching the same trip if tested.
2:40-3:20	Shek	Apply route-segment blockage; show red closure and alternative route. Explain scenario resets to base.
3:20-3:50	Both	Seal destination; show No valid route, disabled Start and unavailable ETA.
3:50-4:30	Both	Apply crowd scenario; explain intensity x confidence x freshness and distance falloff.
4:30-5:00	Shek	Show required 10-second cadence, exact reverse return (or explicitly rerouted return), and identify final telemetry source as future integration.

David's physical-device evidence

Phone-sized browser automation is not physical device sign-off. Fill these rows with what you observe.

Device / OS / browser	Pan & zoom	Layers	Moving position & ETA	Portrait/landscape	Result / tester / time
iPhone / ___ / Safari ___	pending	pending	pending	pending	pending
Android model ___ / OS ___ / Chrome ___	pending	pending	pending	pending	pending
Windows / browser ___	pending	pending	pending	n/a	pending

Shek's telemetry and route evidence

- [ ] Record two received state timestamps approximately 10 seconds apart while moving.

- [ ] Switch to 1 second and observe increased update frequency.

- [ ] Pause stops progression; resume continues rather than restarting.

- [ ] Two viewers see the same shared trip.

- [ ] All six stops load; outbound and exact reverse return are available in clear conditions.

- [ ] Closure avoided in outbound AND return.

- [ ] Sealed destination never starts movement.

- [ ] End of trip is base, with zero ETA and complete status.

- [ ] Network/API loss produces a connection/stale warning.

Defect log

ID	Device / scenario	Expected	Actual	Owner	Fix / retest evidence
D-001					

Suggested card result after observed local tests

“Local simulation demonstrates switchable campus GIS layers, zoom/pan, ETA, shared robot position at 10-second or 1-second cadence, and closure-aware routes; physical device results attached. Hardware telemetry, surveyed missing features and live campus conditions remain integration work.”

Only mark the iOS/Android acceptance criterion complete after device evidence is recorded. Share the ZIP manually with partners; this kit sends no messages or invitations.